Computer Networks (EE353)

Lab 6: Socket Programming (Client-Server Communication Using TCP and UDP in Python)

Arqam Waheed 553514 BSDS 3A 15 October 2026

Setup

Everything was written in Python 3.10.12 and run on Ubuntu from one folder, with a separate terminal for the server and the client. Packets were captured with Wireshark 3.6.2 on the loopback interface **lo**, because both programs run on 127.0.0.1. Two capture files were taken: **lab6_msg.pcapng** (45 packets) for the messaging experiments and **lab6_files.pcapng** (2317 packets) for the file transfers.

Port	Protocol	Used by
5000	TCP	Experiment 1 messaging (+ Group A tasks 1-3)
5001	UDP	Experiment 2 messaging (+ Group A task 4)
5002	TCP	Experiments 5 and 6, simple file sharing
5004	TCP	Experiment 4, byte-stream demonstration
5005	TCP	Group A task 5, calculator
5006	UDP	Group A task 6, echo
6000	TCP	Experiment 8, request-response file server
6001	UDP	Experiment 9, chunked file transfer

The two test files were built with generate_log.py: **sensor_log.txt** (307224 bytes, 300.0 KB, 301 chunks of 1024 bytes) and **notes_log.txt** (102425 bytes, 100.0 KB). Both are kept in server_files/, which is the only folder the file servers will read from.

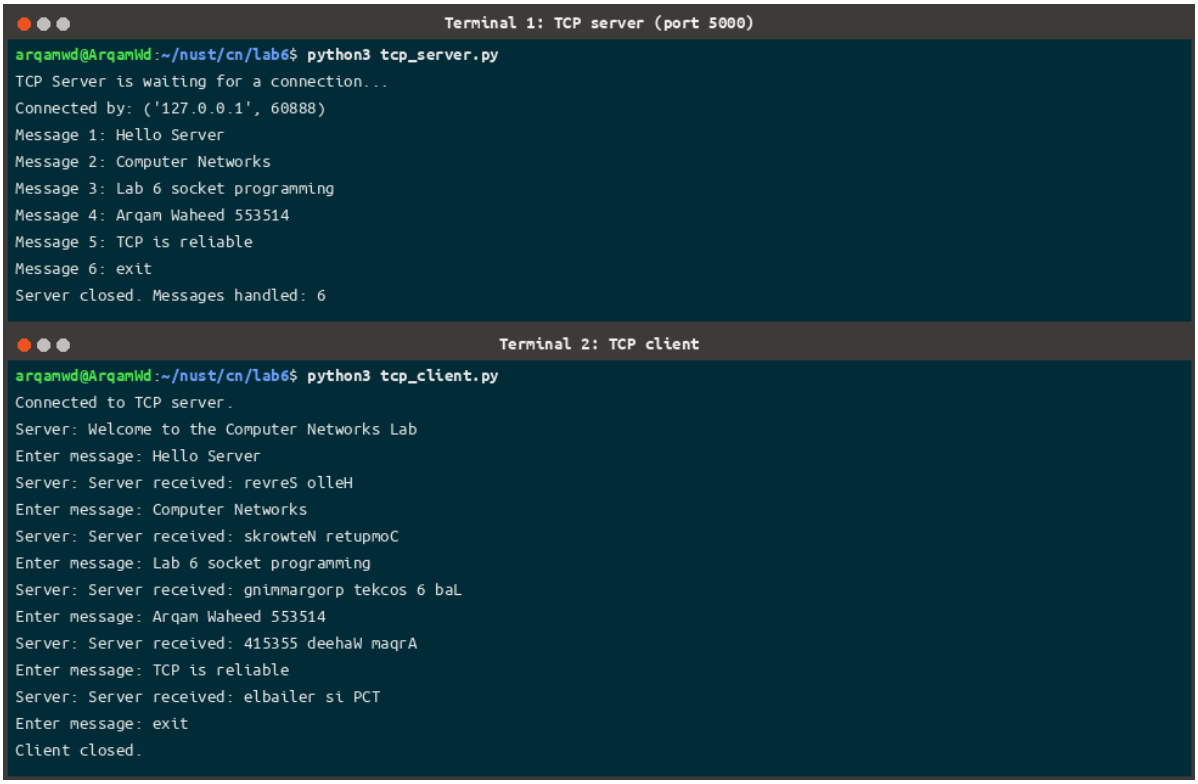
Experiment 1: TCP Client-Server Messaging

The server binds to 127.0.0.1:5000, listens, accepts one client and then loops on recv(). Three of the Group A tasks are already built into it: it sends a welcome line on connect (task 1), numbers every message it receives (task 2) and replies with the text reversed (task 3).

```
conn, addr = server_socket.accept()
print("Connected by:", addr)
conn.send(b"Welcome to the Computer Networks Lab\n")   # Task 1

count = 0
while True:
    data = conn.recv(1024)
    if not data:
        break
    message = data.decode().strip()
    count += 1                                          # Task 2
    print("Message {}: {}".format(count, message))
    if message.lower() == "exit":
        break
    response = "Server received: " + message[::-1]      # Task 3 (reversed)
    conn.send(response.encode())
```

tcp_server.py: the loop that handles one client



Five messages and exit, with the server numbering them and replying in reverse

The client was accepted from port 60888. Every reply came back reversed, for example Computer Networks → skrowteN retupmoC, and the connection closed cleanly after exit.

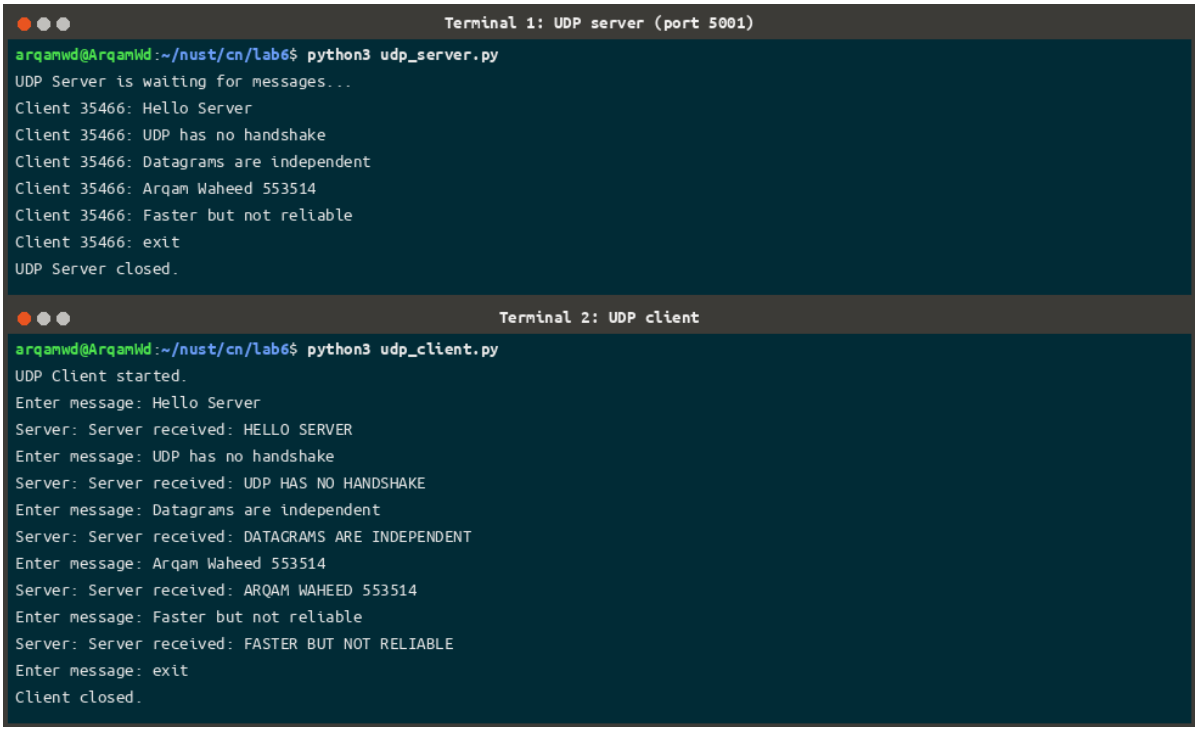
Experiment 2: UDP Client-Server Messaging

The UDP server only calls bind() and then recvfrom(); there is no listen(), accept() or connect(). recvfrom() returns the data and the sender's address, which is what the reply is sent back to. Group A task 4 (reply in uppercase) is built into it.

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server_socket.bind((HOST, PORT))

while True:
    data, addr = server_socket.recvfrom(1024)
    message = data.decode().strip()
    print("Client {}: {}".format(addr[1], message))
    if message.lower() == "exit":
        break
    response = "Server received: " + message.upper()     # Task 4
    server_socket.sendto(response.encode(), addr)
```

udp_server.py

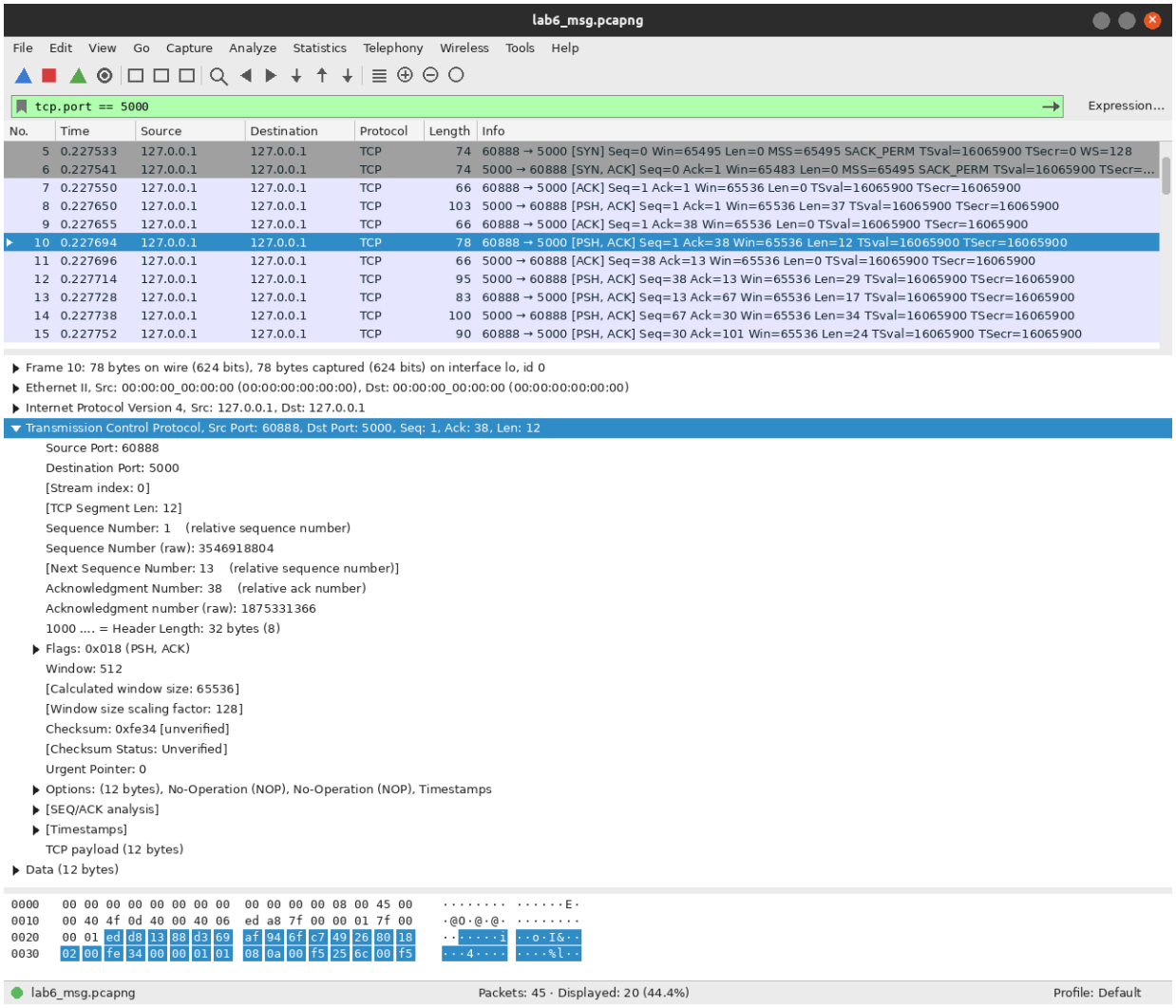


The same five messages over UDP, answered in uppercase

The client used port 35466. Nothing is exchanged before the first message or after the last one; the client just stops.

Experiment 3: Observing the Traffic in Wireshark

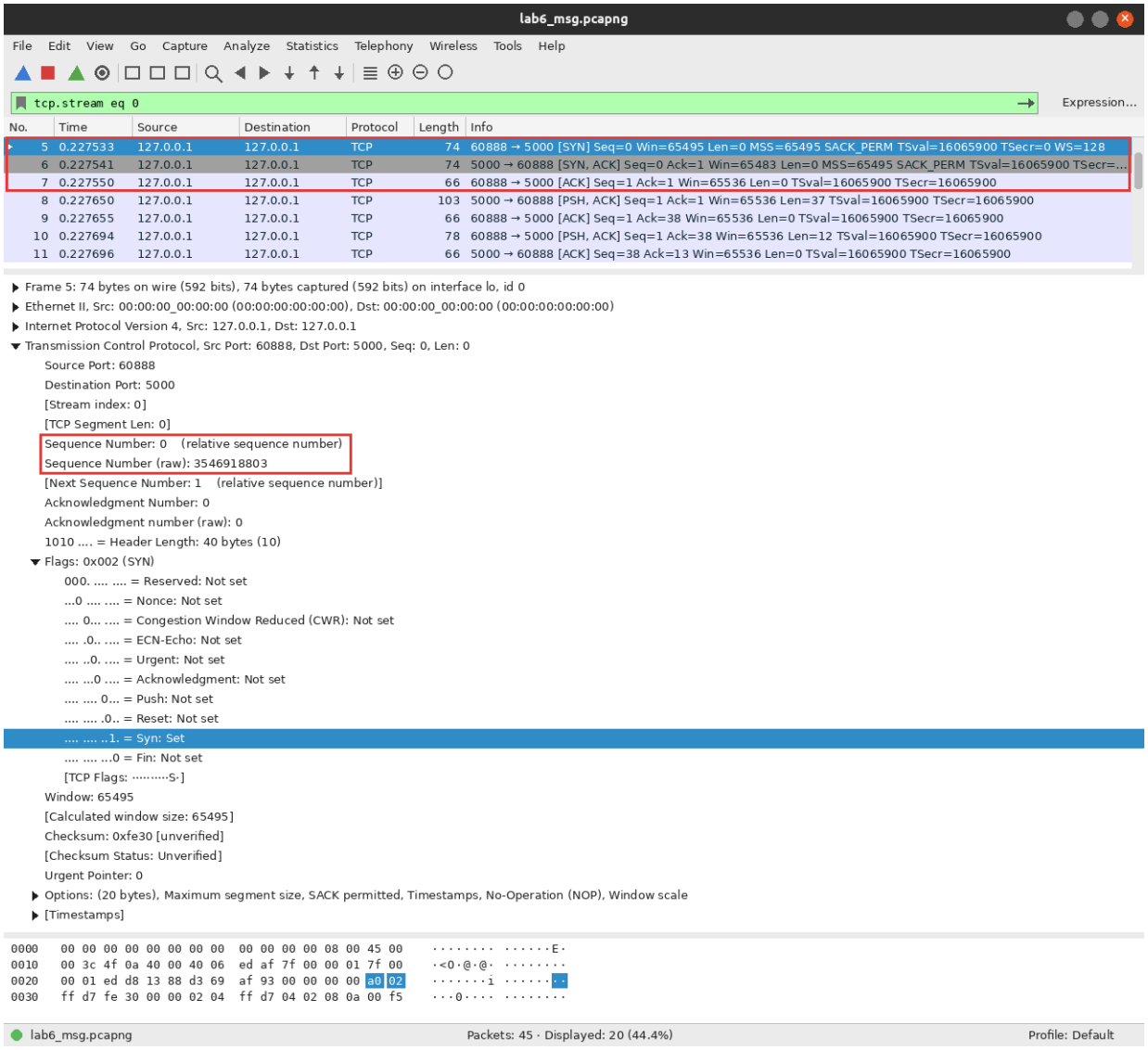
Wireshark was started on the loopback interface before running Experiments 1 and 2, so both conversations are in lab6_msg.pcapng. The filter **tcp.port == 5000** shows 20 of the 45 packets and **udp.port == 5001** shows 11 more. The other 14 are DNS lookups from other programs that the loopback interface carried while I was capturing.



Filter tcp.port == 5000, frame 10 selected: the client's first message

Frame 10 carries 12 bytes (Hello Server) from 127.0.0.1:60888 to 127.0.0.1:5000. The TCP header is 32 bytes (20 plus 12 bytes of timestamp options) and the payload follows it.

Three-way handshake

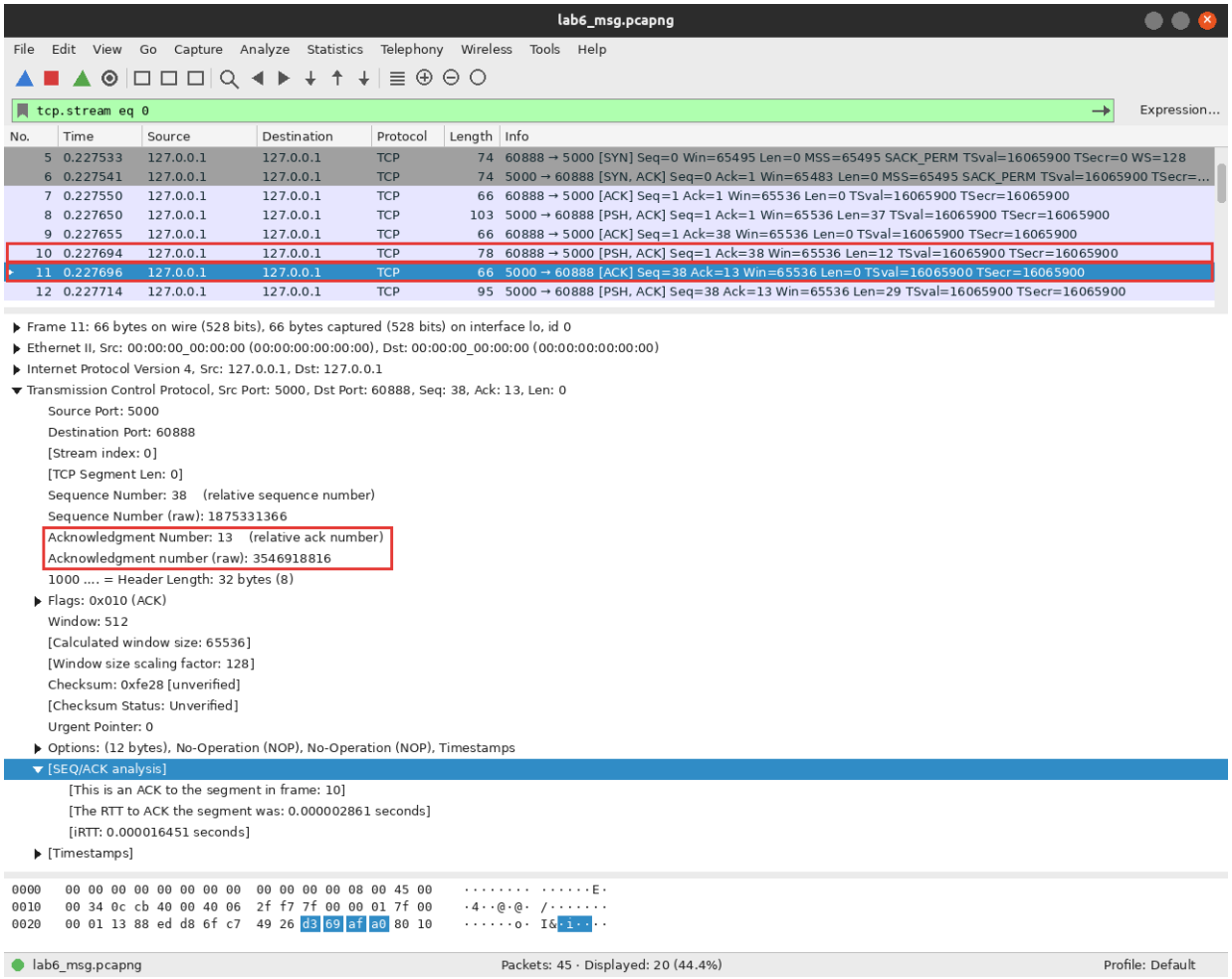


Frames 5-7: SYN, SYN/ACK, ACK on tcp.stream eq 0

Packet	Frame	Ports	Flags	Seq / Ack
SYN	5	60888 → 5000	0x002 (SYN)	Seq 0 (raw 3546918803)
SYN/ACK	6	5000 → 60888	0x012 (SYN, ACK)	Seq 0, Ack 1
ACK	7	60888 → 5000	0x010 (ACK)	Seq 1, Ack 1

Only after these three packets does any data move. The SYN also advertises MSS=65495, SACK_PERM and a window scale of 128, which is normal for loopback. The initial round trip (iRTT) was 0.000016 s.

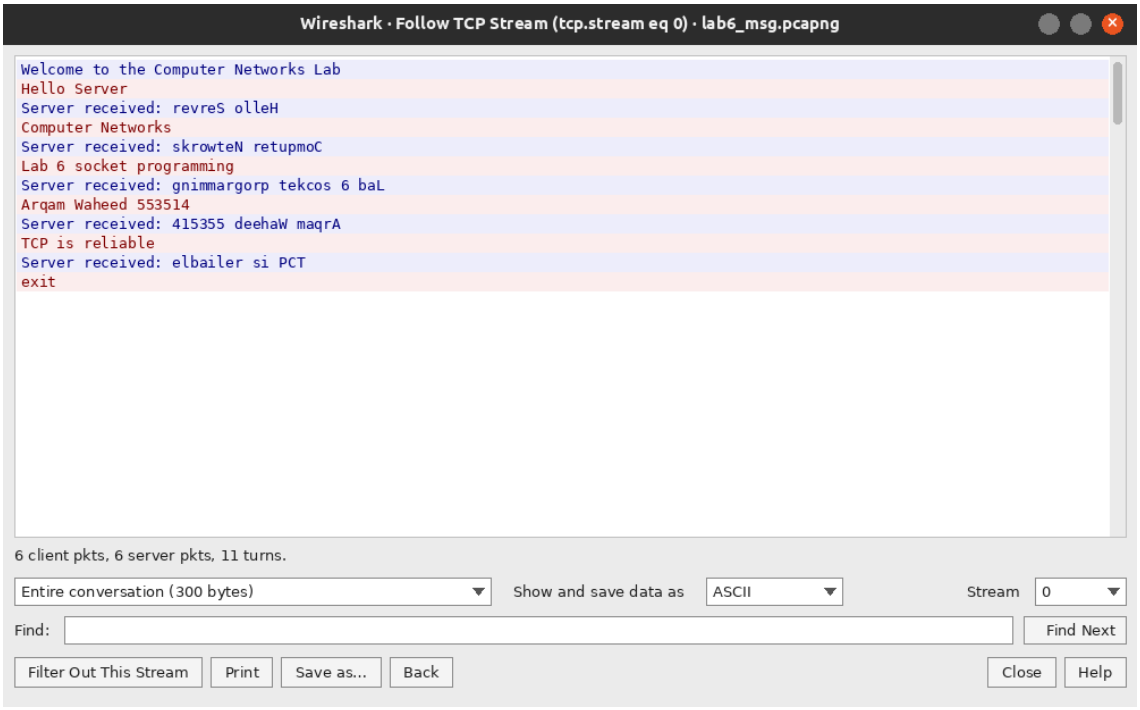
Sequence and acknowledgment numbers



Frame 11 is the server's ACK for the 12 data bytes in frame 10

Field	Value
Data packet	Frame 10, client → server, 12 bytes
Sequence number	1
Next sequence number	13 (1 + 12)
ACK packet	Frame 11, server → client
Acknowledgment number	13 (the next byte the server expects)
Time between them	0.0000029 s

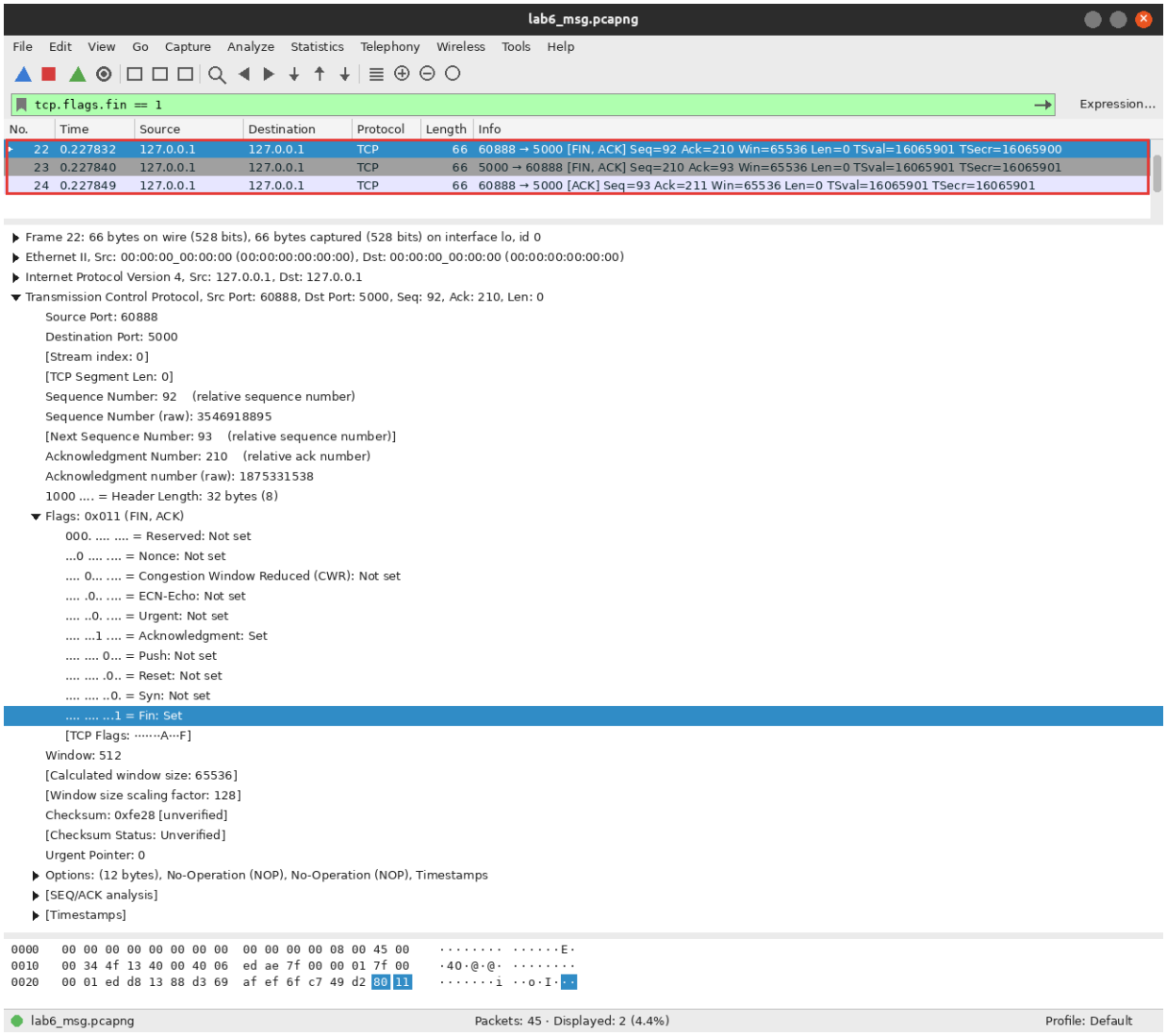
Follow TCP Stream



Analyze → Follow → TCP Stream on frame 10 (6 client packets, 6 server packets, 11 turns, 300 bytes)

Red is what my client sent, blue is the server's reply. The whole conversation is readable because nothing is encrypted; the welcome line, the five messages and the five reversed replies are all there.

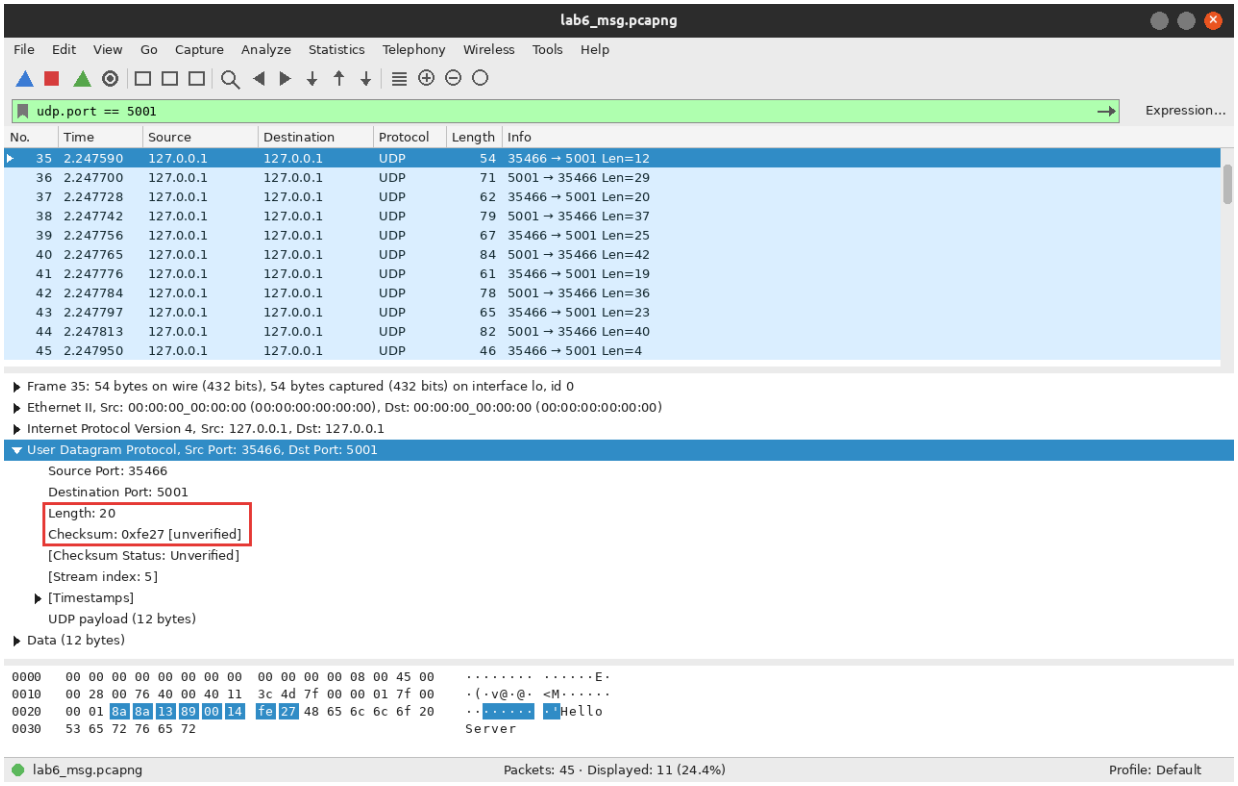
Connection termination



Filter tcp.flags.fin == 1, frames 22-24

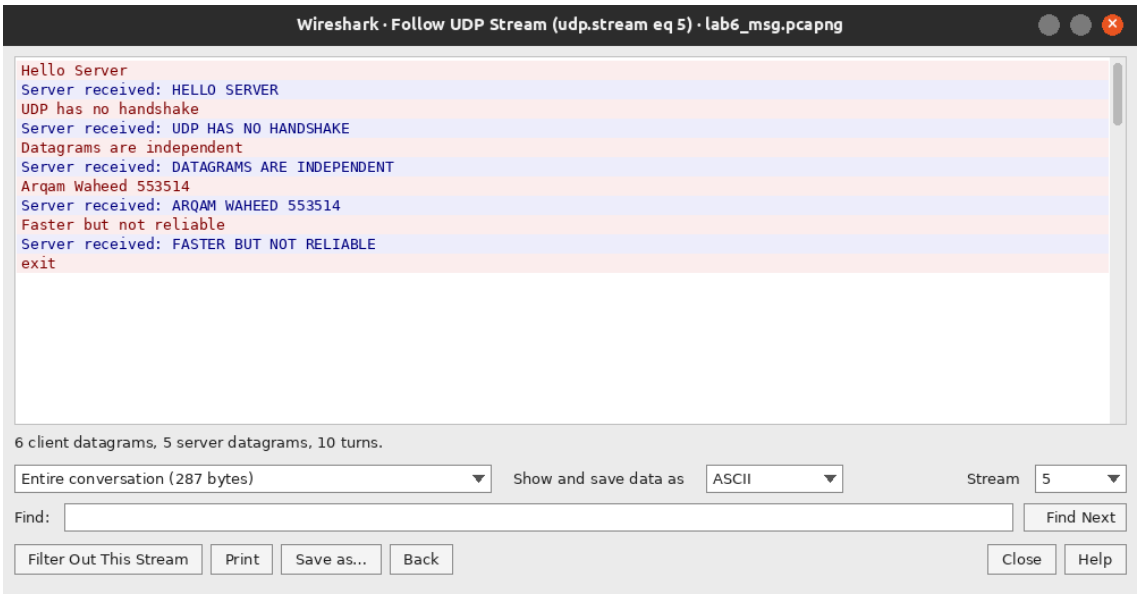
The client sent FIN, ACK (frame 22) after exit, the server answered with its own FIN, ACK (frame 23) and the client acknowledged it (frame 24). TCP is full duplex, so each direction has to be closed separately, which is why FINs travel both ways.

UDP traffic



Filter udp.port == 5001, frame 35 selected

Frame 35 is the first message. The UDP header has only four fields: source port 35466, destination port 5001, Length 20 (8 bytes of header plus 12 of data) and Checksum 0xfe27. Nothing was sent before it and nothing is sent after the last one.



Follow → UDP Stream: 6 client datagrams, 5 server datagrams, 287 bytes

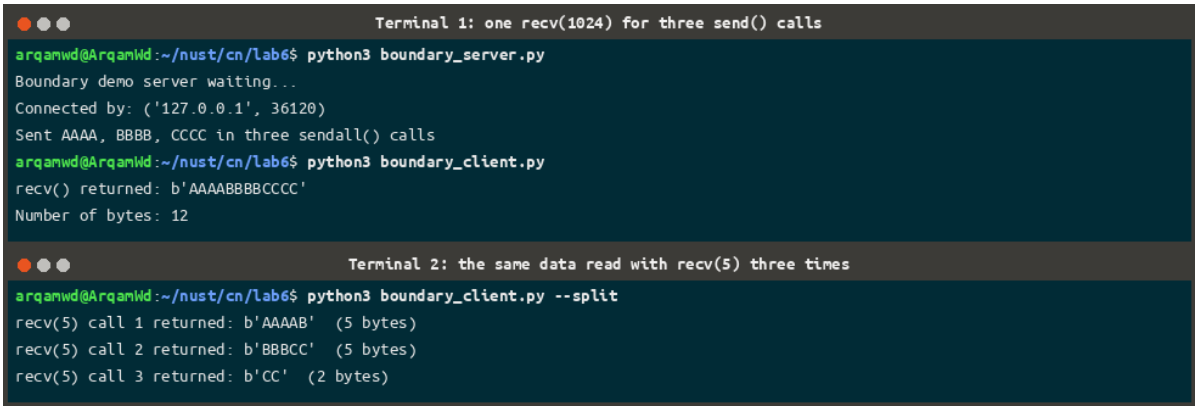
Comparison table (Group B)

Observation	TCP (port 5000)	UDP (port 5001)
Packets before the first message	3 (SYN, SYN/ACK, ACK)	none, data is sent immediately
Packets for one message + reply	data (PSH, ACK) + ACK, then the reply	one datagram each way
Header fields visible	ports, seq, ack, flags, window, checksum, options	ports, length, checksum
Header size here	32 bytes (20 + 12 options)	8 bytes
End of communication	FIN, ACK / FIN, ACK / ACK (frames 18-20)	nothing is sent
Total packets for 5 messages	20	11
My ports	client 60888, server 5000	client 35466, server 5001

The same five messages needed 20 TCP packets but only 11 UDP datagrams, because TCP adds a handshake, acknowledgments and a close sequence that UDP does not have.

Experiment 4: TCP Is a Byte Stream

The server makes three separate sendall() calls (AAAA, BBBB, CCCC) and the client reads once with recv(1024). Then the same data is read with recv(5) three times.

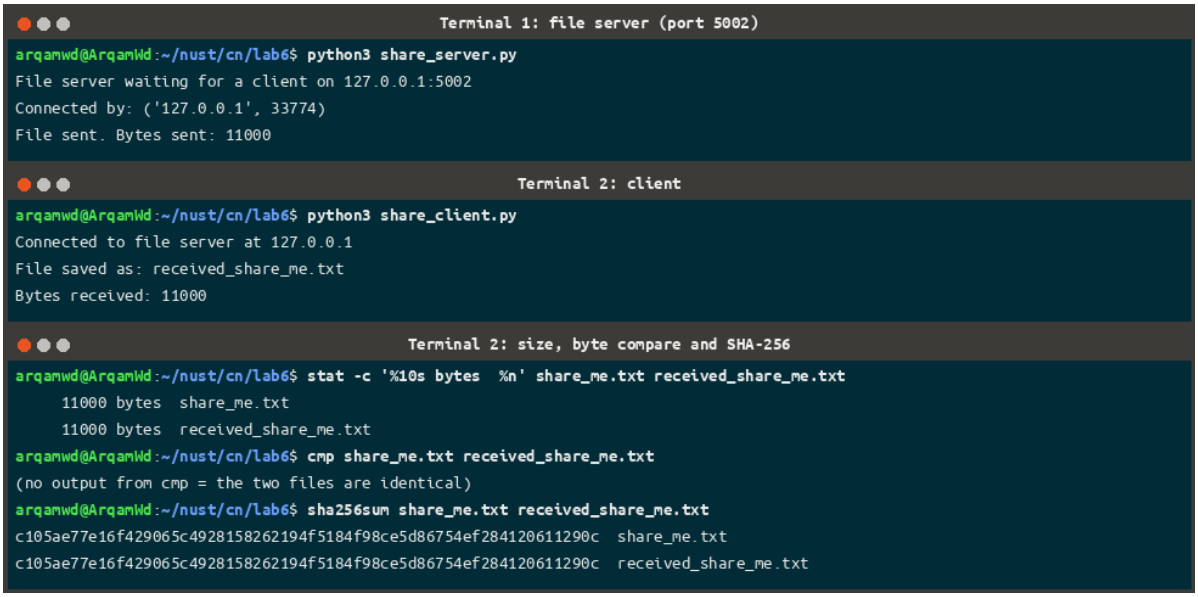


One recv() returned all twelve bytes; recv(5) split them in a completely different place

The first run printed b'AAAABBBBCCCC', 12 bytes from a single recv(), so three messages were merged. The second run returned b'AAAAB', b'BBBCC' and b'CC', so one message was split across two reads. TCP keeps the order of the bytes, but the boundaries between send() calls are gone. The file client in Experiment 8 therefore has to be told the size first and then loop until it has counted every byte.

Experiment 5: File Sharing Over TCP

share_me.txt (11000 bytes) is read in 1024-byte chunks and sent with sendall(); the client writes every piece it receives until recv() returns b"", which means the server closed the connection.

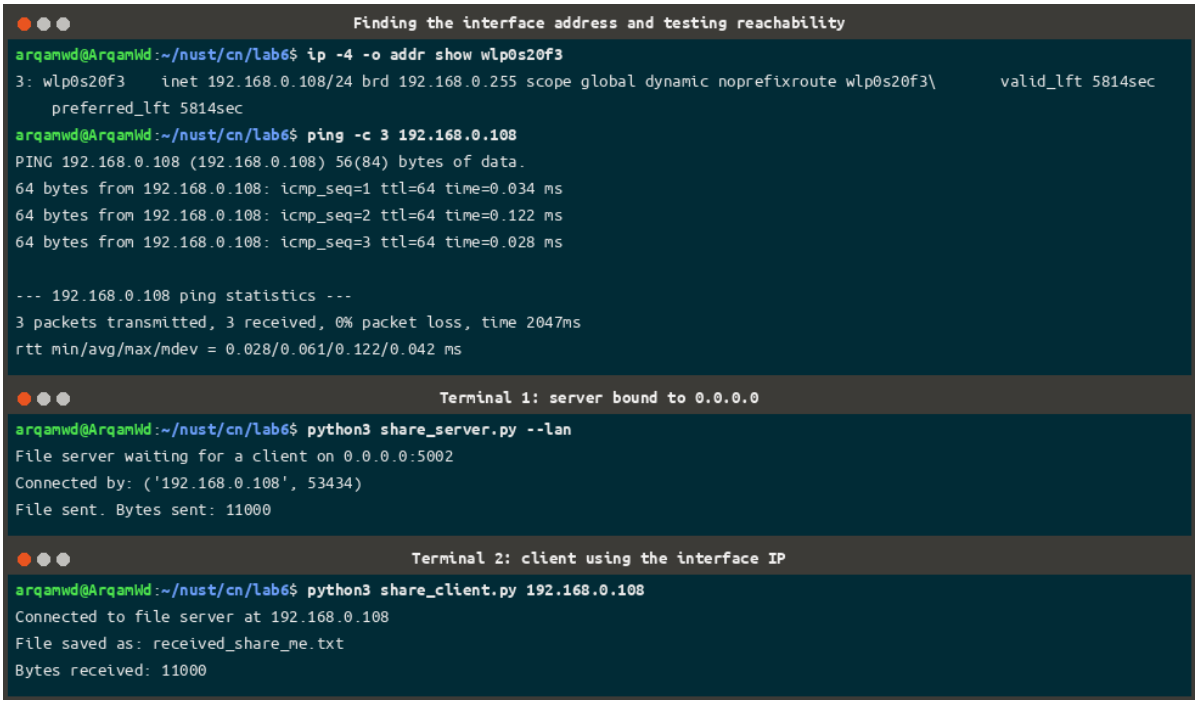


11000 bytes sent and received, identical size, no difference reported by cmp, matching SHA-256

Both files are 11000 bytes, cmp printed nothing (no difference) and the SHA-256 of both is c105ae77e16f4290…0611290c. The weakness of this version is that the client only knows the file ended because the connection closed: a broken connection would look exactly the same as a finished transfer.

Experiment 6 (Optional): Sharing Over a Real Interface Address

I did not have a partner machine, so instead of the loopback address I ran the same programs over the real Wi-Fi interface address of my laptop (wlp0s20f3, 10.18.162.246). The server was bound to 0.0.0.0 and the client was given the interface IP, which is the only change Experiment 6 asks for; the packets still stay inside one machine, so this shows the addressing part but not a second host.



ip -4 addr, ping to the interface address, then the same transfer using 10.18.162.246

The fingerprints of share_me.txt and received_share_me.txt matched again, so the copy is exact.

Experiment 7: Test File, Integrity Check and Timing



Creating the 300 KB and 100 KB logs and fingerprinting them

File	Size	SHA-256 (first 16 hex digits)	Hash time
sensor_log.txt	307224 bytes (300.0 KB)	fc33991fe8d1d508…	0.000270 s
notes_log.txt	102425 bytes (100.0	4551da1280478a6e…	0.000129 s

	KB)		

A SHA-256 hash is a fingerprint: if one byte changes, the whole value changes, so comparing hashes proves the contents are the same, which the length on its own cannot. Throughput is the file size in KB divided by the elapsed seconds, and time.perf_counter() is used for the timing because it is a high-resolution clock.

Experiment 8: Request-Response File Server Over TCP

This is the version that fixes the weakness of Experiment 5. The client asks for a file by name, the server answers with a header that announces the size and the SHA-256, and only then sends the bytes. Because the client knows the length, it can tell a finished transfer from a broken one.

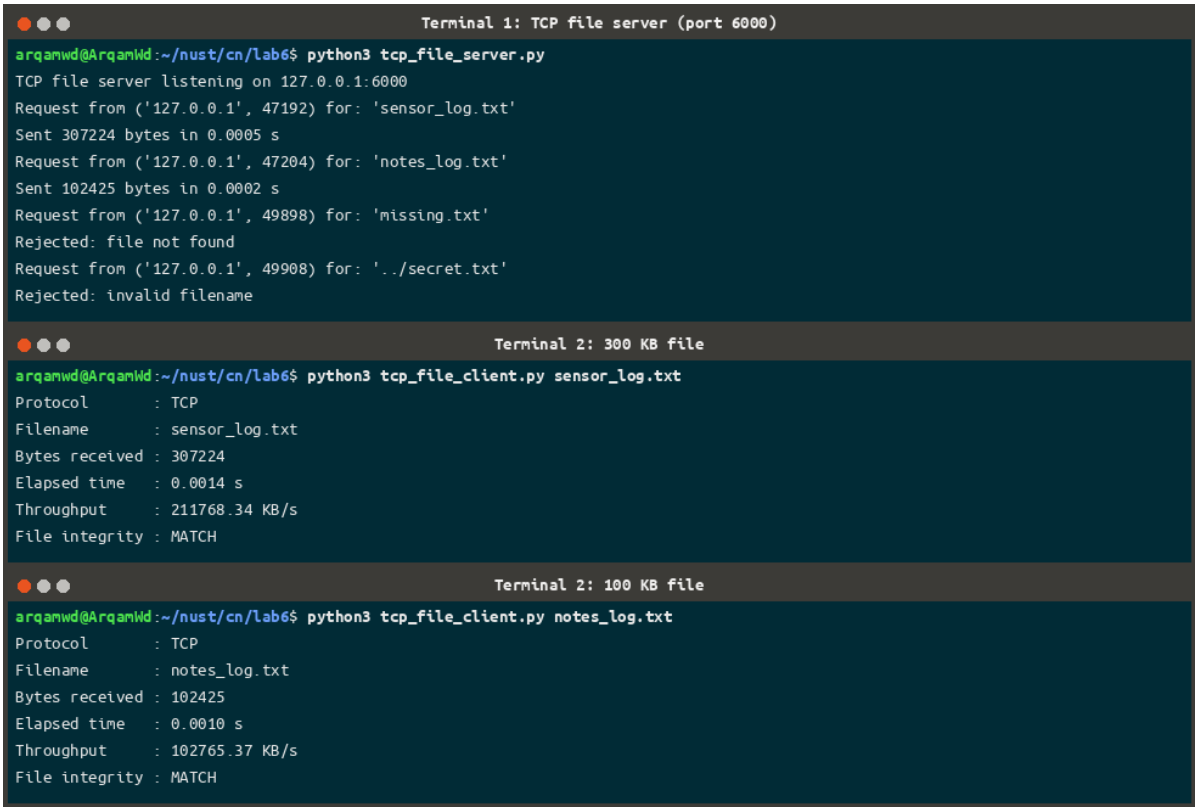
Step	Direction	Message
1	client → server	filename followed by a newline
2	server → client	OK <size> <sha256>\n or ERROR <reason>\n
3	server → client	exactly <size> bytes, sent in 1024-byte chunks
4	client	counts the bytes, saves the file, compares the hash

```
    size, expected_hash = int(reply[1]), reply[2]
    received = 0
    sha = hashlib.sha256()

    with open(save_as, "wb") as f:
        while received < size:
            data = client.recv(min(BUFFER_SIZE, size - received))
            if not data:
                raise ConnectionError("Connection closed early: got {} of {} bytes"
                                      .format(received, size))
            f.write(data)
            sha.update(data)
            received += len(data)
```

tcp_file_client.py: the exact-length receive loop

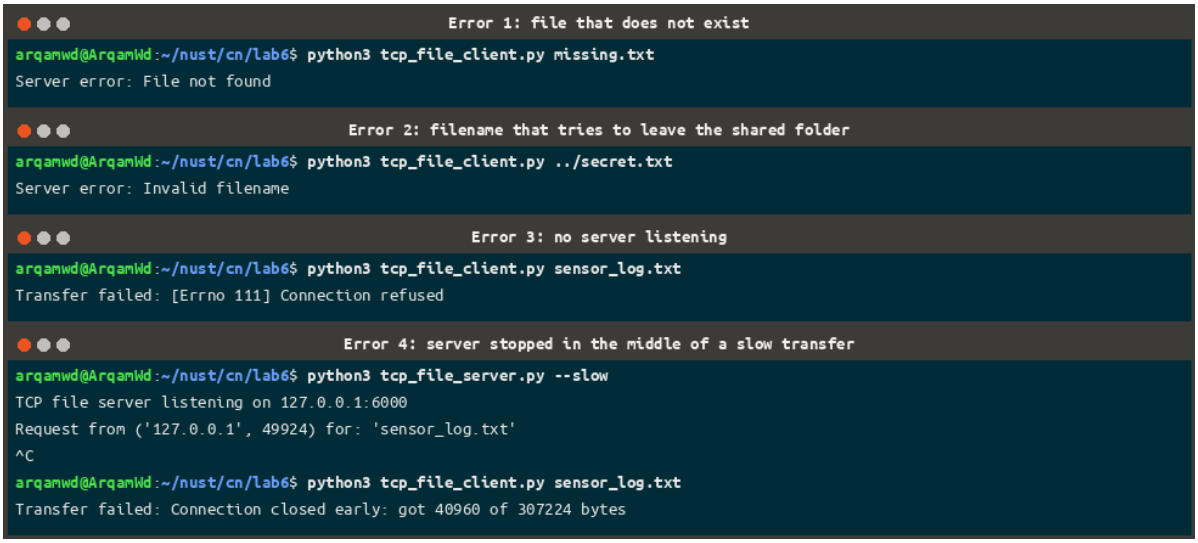
recv(min(BUFFER_SIZE, size - received)) never reads past the end of the file, and the loop keeps going until every byte is counted, which is what makes it safe against the merging and splitting seen in Experiment 4. The server also refuses any filename that is not a plain name, so ../secret.txt cannot escape server_files/.



The 300 KB and the 100 KB file, both verified with MATCH

File	Bytes	Elapsed	Throughput	Integrity
sensor_log.txt	307224	0.0014 s	211768.34 KB/s	MATCH
notes_log.txt	102425	0.0010 s	102765.37 KB/s	MATCH

Error handling



The four failure cases, all handled without crashing the server

Test	What the client printed
missing.txt	Server error: File not found
../secret.txt	Server error: Invalid filename
server not running	Transfer failed: [Errno 111] Connection refused
server stopped mid-transfer	Transfer failed: Connection closed early: got 40960 of 307224 bytes

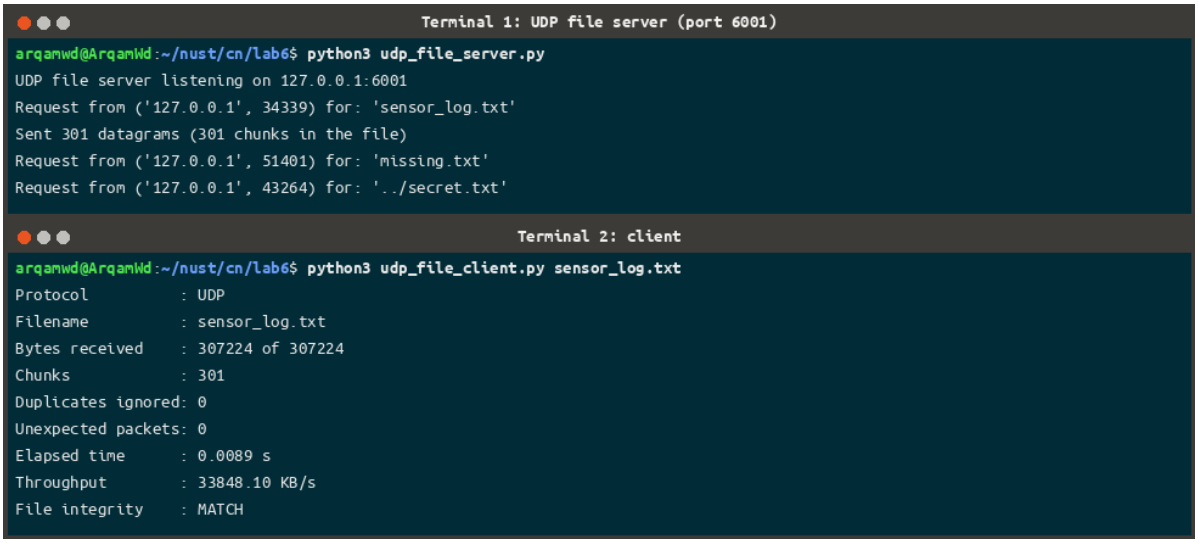
Without the size header the client would have saved those 40960 bytes and treated them as the whole file.

Experiment 9: File Transfer Over UDP

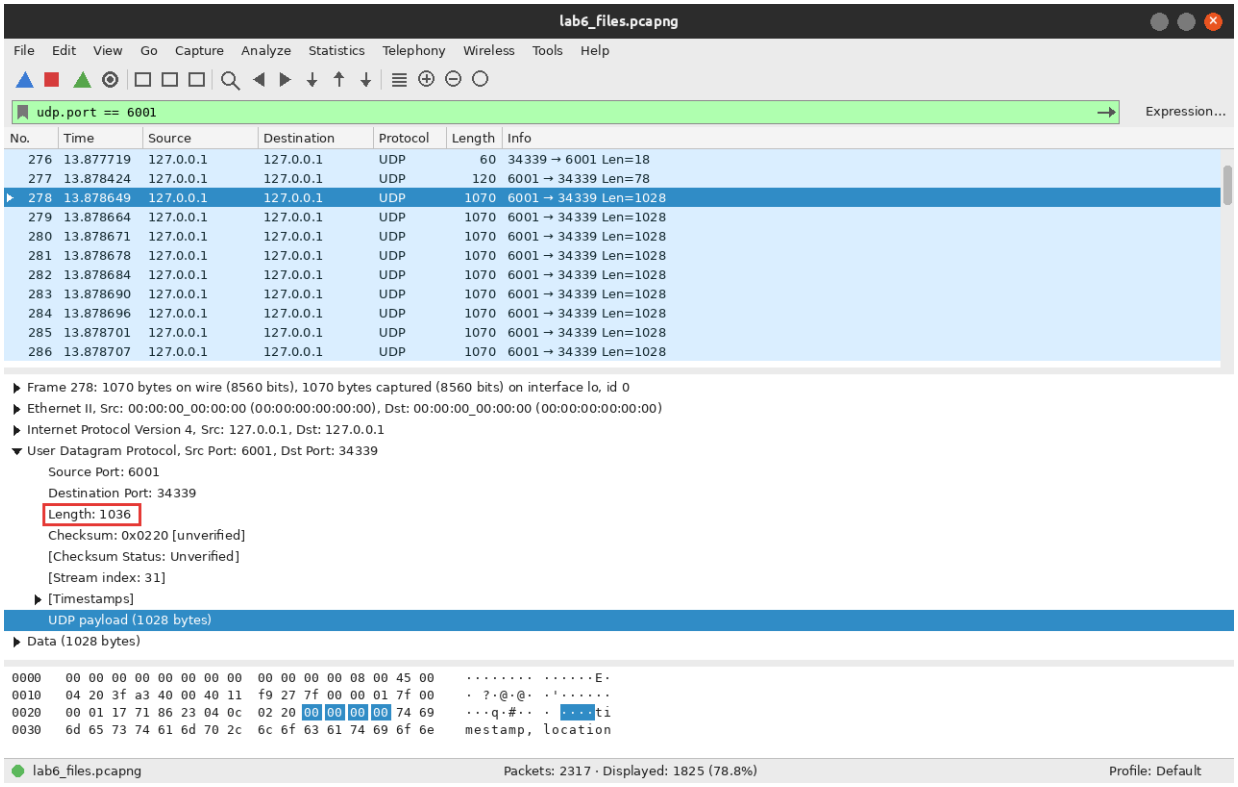
UDP has no connection and no ordering, so the application has to add everything itself. Each datagram carries a 4-byte sequence number packed with struct.pack("!I", seq) followed by up to 1024 bytes of file data. 307224 bytes therefore need 301 chunks. The client stores them in a dictionary keyed by sequence number, so a repeated number is a duplicate and sorting the keys puts the file back in order.

```
    for count, seq in enumerate(order, start=1):
        payload = data[seq * CHUNK_SIZE:(seq + 1) * CHUNK_SIZE]
        server.sendto(struct.pack("!I", seq) + payload, addr)
        sent += 1
        if count % PACE_EVERY == 0:
            time.sleep(0.001)      # without this the receive buffer overflows
```

udp_file_server.py: sending the numbered chunks



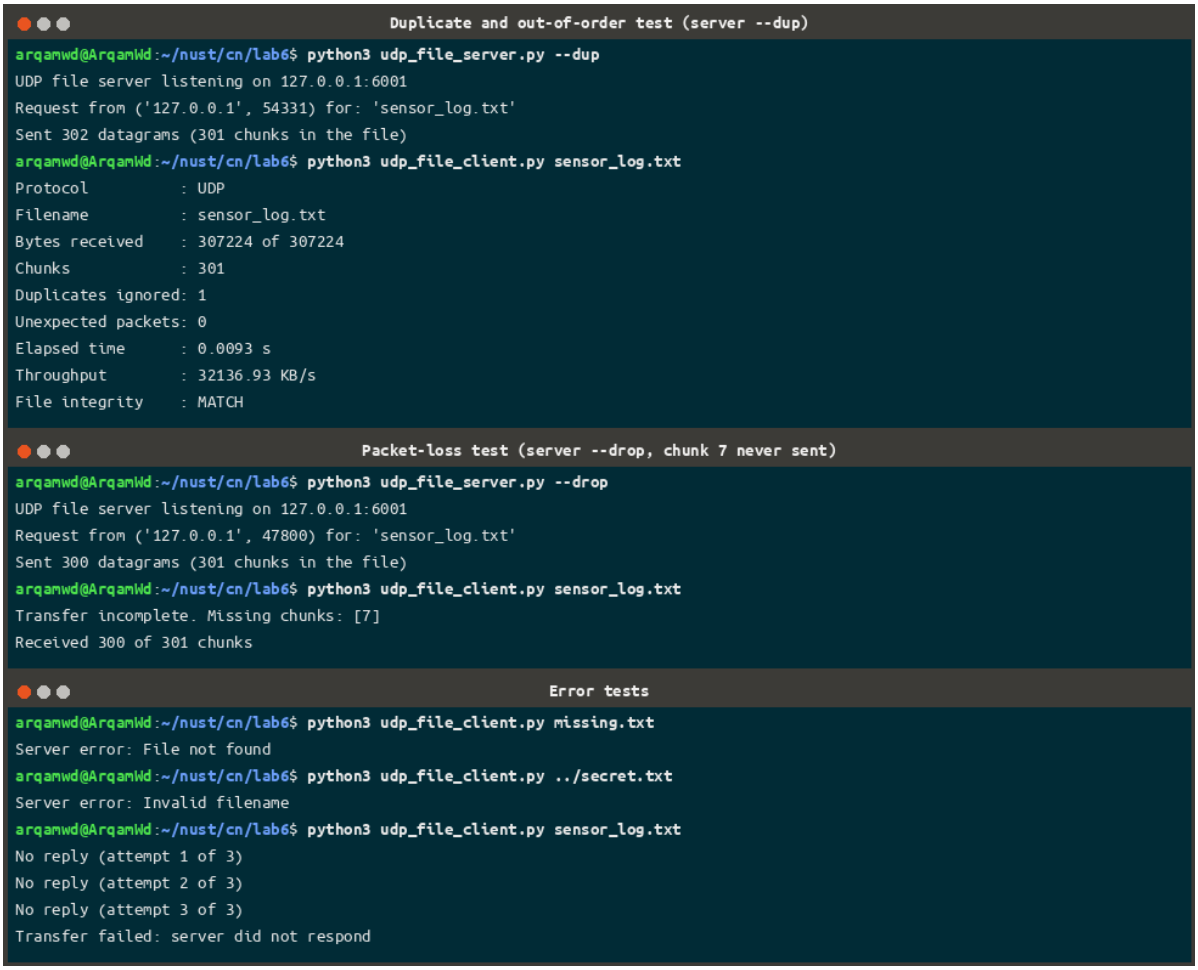
301 chunks, no duplicates, no missing chunks, MATCH



Filter udp.port == 6001: the GET, the OK header and the data datagrams. The four highlighted bytes 00 00 00 00 are the sequence number of chunk 0

Frame 276 is GET sensor_log.txt (18 bytes), frame 277 is the reply OK 307224 301 fc33991f... (78 bytes) and from frame 278 onwards every datagram is 1028 bytes: 4 bytes of sequence number plus 1024 bytes of data.

Duplicate, out-of-order and packet-loss tests



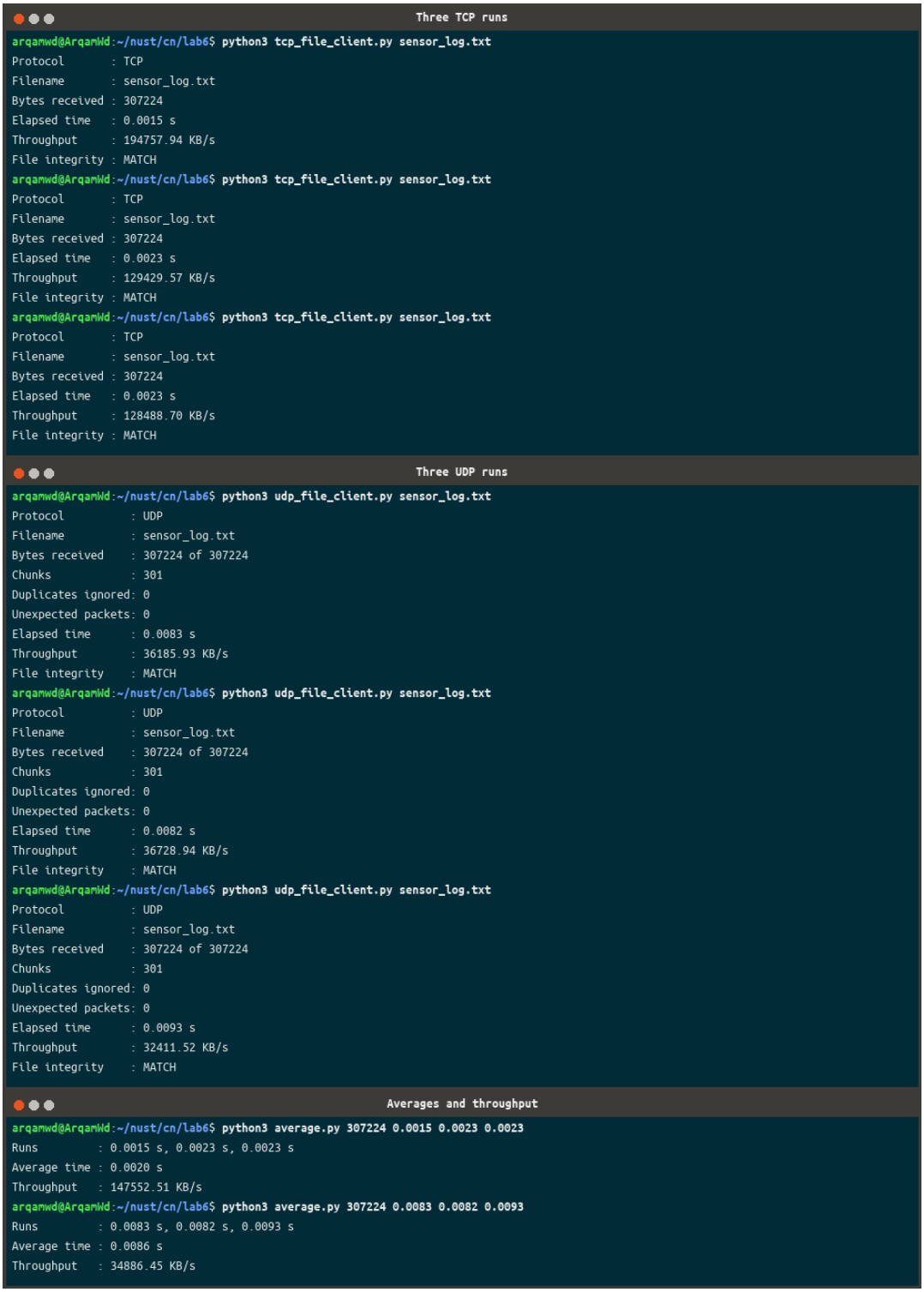
--dup duplicates chunk 3 and swaps 10 and 11; --drop never sends chunk 7

Test	Result
Normal transfer	301 chunks, 0 duplicates, MATCH
Duplicate + out of order	302 datagrams sent, Duplicates ignored: 1, still MATCH
Chunk 7 dropped	Transfer incomplete. Missing chunks: [7] (300 of 301 received)
missing.txt / ../secret.txt	Server error: File not found / Invalid filename
No server running	three attempts, then Transfer failed: server did not respond

The duplicate and the swap caused no damage because the sequence numbers put everything back in place, but the dropped chunk was never recovered: after 2 seconds of silence the client reported the gap and gave up. TCP would have retransmitted it automatically. To repair it over UDP the client would have to ask for the missing numbers again, which is the optional challenge in the manual.

Experiment 10: Performance Comparison

Both servers were started with every test switch off and the same 300 KB file was fetched three times with each client.



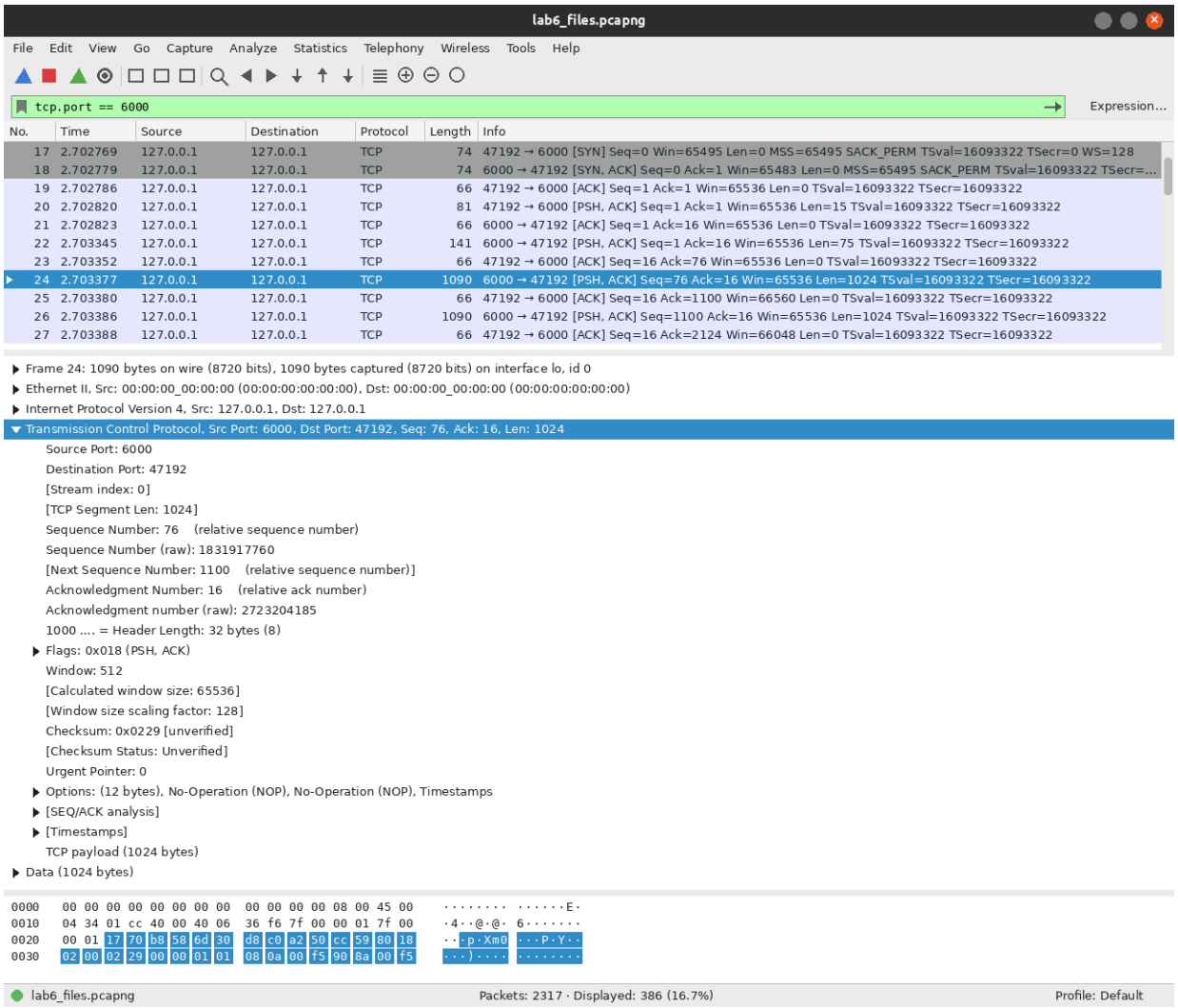
Three runs per protocol and the averages

Protocol	Run 1 (s)	Run 2 (s)	Run 3 (s)	Average (s)	Throughput (KB/s)
TCP	0.0015	0.0023	0.0023	0.0020	147552.51
UDP	0.0083	0.0082	0.0093	0.0086	34886.45

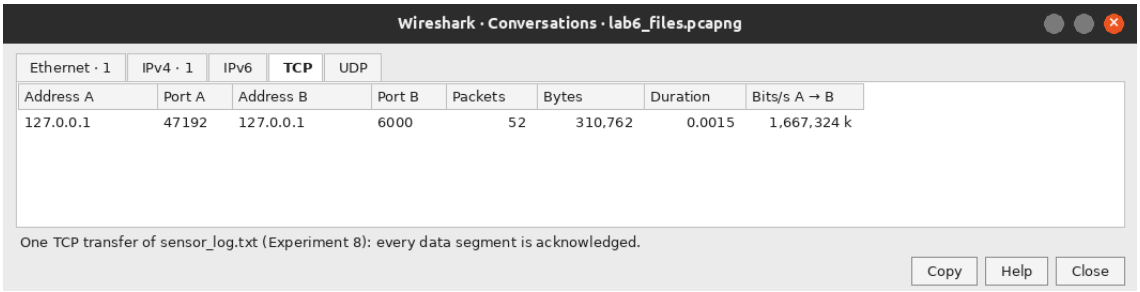
Comparison

On this machine TCP finished the 300 KB file about four times faster than my UDP program, which is the opposite of what the textbook comparison of the two protocols would suggest. The reason is that the numbers measure my two programs, not the protocols on a real network. Everything runs over loopback, where there is no real loss or congestion, so TCP pays almost nothing for its reliability and the kernel can hand it very large segments. My UDP sender, on the other hand, makes 301 separate sendto() calls and sleeps for 1 ms after every 50 of them to stop the receiver's buffer from overflowing, and that pacing alone accounts for about 6 ms of the 8.6 ms. On a real lossy link the picture would change, because TCP would start retransmitting and backing off while UDP would simply keep sending. TCP handed me ordering, retransmission, flow control and a clear end of file for nothing, and over UDP I built the same things by hand out of sequence numbers, a chunk count, duplicate detection and a timeout.

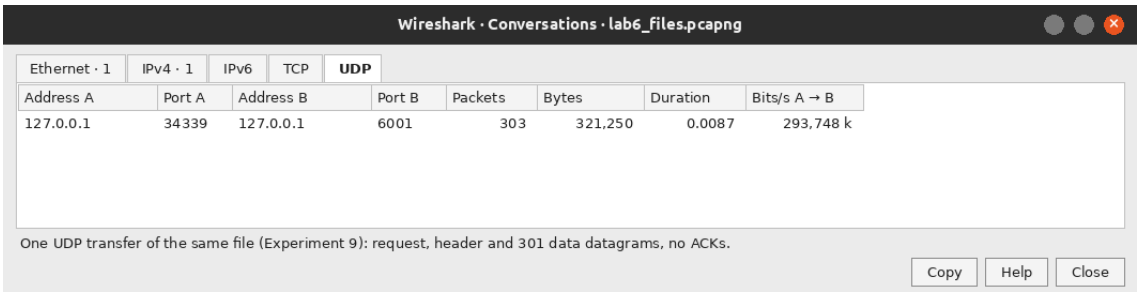
Wireshark comparison of the two transfers



Filter tcp.port == 6000: the file arrives in a few very large segments on loopback



Statistics → Conversations, TCP tab: one transfer of sensor_log.txt



The UDP tab for the same file

Measure	**TCP (port 6000)**	**UDP (port 6001)**
Packets for one 300 KB transfer	52	303
Bytes on the wire	310,762	321,250
Acknowledgments	yes, the client ACKs every segment	none
Largest data unit seen	49280 bytes (segments are merged)	1028 bytes (4 + 1024)
Connection packets	SYN, SYN/ACK, ACK and FINs	none

TCP needed far fewer packets because loopback lets it send huge segments and merge my 1024-byte writes, while UDP has to keep every chunk as its own datagram. The byte totals are close, but only the UDP capture shows one packet per application chunk. On a real network TCP would be limited to about 1500 bytes per segment, so the packet counts would be much closer.

Lab Tasks: Group A (Messaging)

Tasks 1, 2 and 3 are in tcp_server.py and task 4 is in udp_server.py; their screenshots are the ones already shown for Experiments 1 and 2. Tasks 5 and 6 are separate programs.

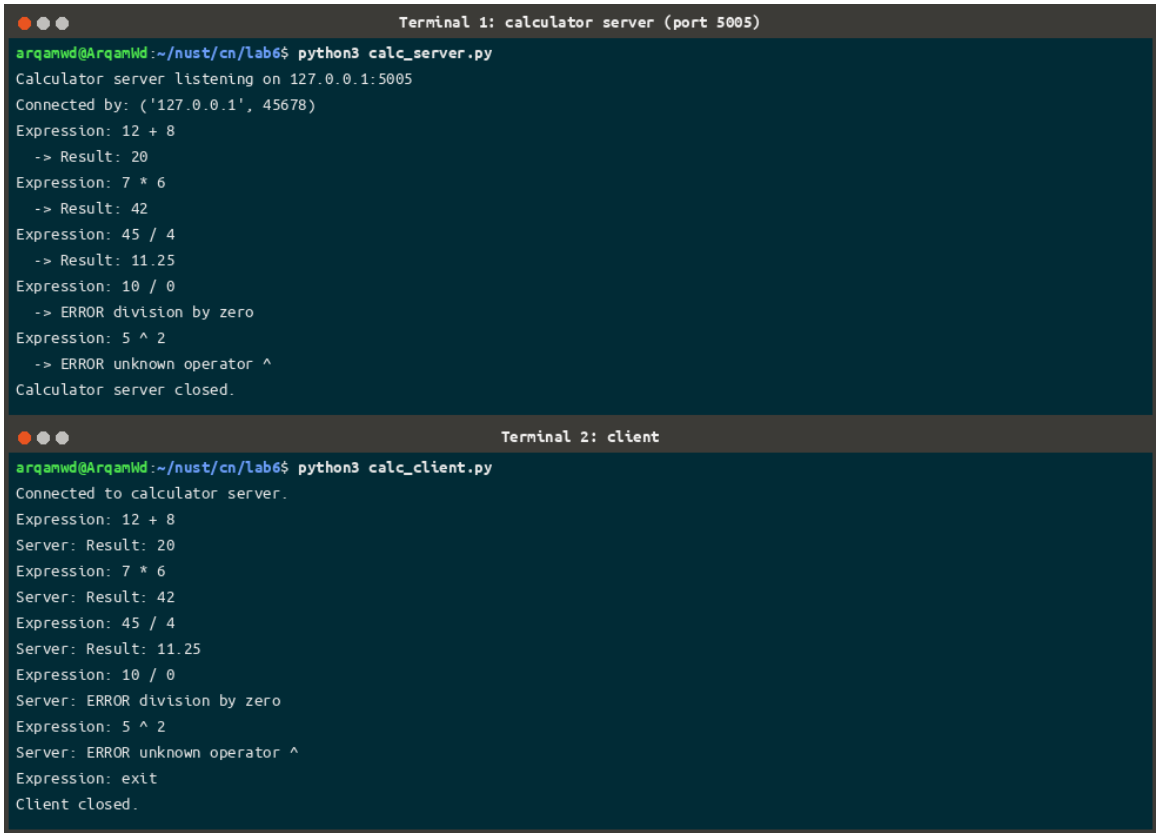
Task	Change	Evidence
1. Welcome message	conn.send(b"Welcome to the Computer Networks Lab\n") right after accept()	first line of the client in Experiment 1
2. Message counter	count += 1 before printing, so the server prints Message 1, Message 2, ...	server pane of Experiment 1
3. Reverse server	response = "Server received: " + message[::-1]	Computer Networks → skrowteN retupmoC
4. Uppercase UDP	response = "Server received: " + message.upper()	server pane of Experiment 2
5. TCP calculator	new pair calc_server.py / calc_client.py on port 5005	screenshot below
6. UDP echo	new pair echo_server.py / echo_client.py on port 5006	screenshot below

Task 5: TCP calculator (port 5005)

The client sends one line of the form number operator number and the server returns the answer. Division by zero and unknown operators are answered with an ERROR line instead of crashing the server.

```
    parts = text.split()
    if len(parts) != 3:
        answer = "ERROR send: number operator number"
    else:
        try:
            a, op, b = float(parts[0]), parts[1], float(parts[2])
            if op == "+":
                answer = "Result: {:g}".format(a + b)
            ...
            elif op == "/":
                answer = "ERROR division by zero" if b == 0 else "Result: {:g}".format(a / b)
            else:
                answer = "ERROR unknown operator " + op
        except ValueError:
            answer = "ERROR those are not numbers"
```

calc_server.py



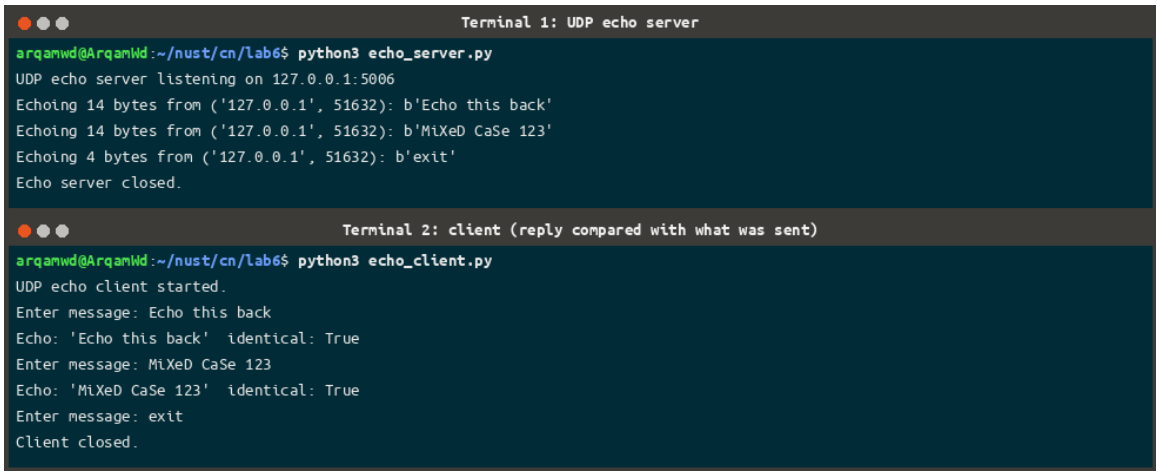
12 + 8 = 20, 7 * 6 = 42, 45 / 4 = 11.25, and the two error cases

Task 6: UDP echo (port 5006)

The echo server sends back exactly the bytes it received, with nothing added and nothing changed, and the client compares the reply with what it sent.

```
    data, addr = server_socket.recvfrom(1024)
    print("Echoing {} bytes from {}: {!r}".format(len(data), addr, data))
    server_socket.sendto(data, addr)          # exactly the same bytes, nothing added
```

echo_server.py



Both replies came back identical, including the mixed-case text

Lab Tasks: Groups B and C

Group B (packet details and the TCP/UDP comparison) is answered by the Wireshark section of Experiment 3, including the source and destination addresses and ports, the handshake flags, the packet carrying the first message and the UDP length and checksum.

Group C is answered by Experiments 8, 9 and 10: two files of different sizes transferred over TCP with MATCH, the three error screenshots, the UDP transfer with its duplicate/out-of-order and packet-loss tests, and the performance table with the comparison paragraph. The last check is that both received copies are byte-identical to the original:



The TCP copy, the UDP copy and the original all have the same SHA-256 and the same size

Review Questions

1. What is the difference between connection-oriented and connectionless communication? Give one example of each. Connection-oriented means the two sides set up a connection before any data is sent and tear it down afterwards, and the protocol guarantees ordered, reliable delivery. TCP works this way, and my file server on port 6000 uses it. Connectionless means every message is sent on its own with no setup and no guarantee, which is UDP, as used by my messaging server on port 5001.

2. Why does a server call bind() while a client usually does not? The server has to be reachable at an address the client already knows, so it binds itself to a fixed port (5000, 6000 ...). The client only needs some free port to receive the answer on, so it lets the operating system pick an ephemeral one, which was 60888 and 35466 in my captures.

3. What do listen() and accept() do? What does accept() return? listen() turns the socket into a passive one that collects incoming connection requests in a queue. accept() blocks until one arrives and returns a pair: a brand-new socket used only for that client, and the client's address. The original socket stays free to accept the next client.

4. Why can one recv() return fewer bytes than were sent, or several messages joined together? How does the TCP file client handle this? TCP is a stream of bytes, not of messages: the kernel is free to merge what several send() calls wrote or to deliver only part of it, which is exactly what Experiment 4 showed. The file client handles it by learning the size from the header first and then looping with recv(min(BUFFER_SIZE, size - received)) until it has counted every byte.

5. Why does the TCP file server send the file size before the file data? So the client knows where the file ends without waiting for the connection to close. That is what lets it report Connection closed early: got 40960 of 307224 bytes instead of silently saving a half file.

6. What is the difference between 127.0.0.1 and 0.0.0.0? 127.0.0.1 is the loopback address, which always means this computer, so only local programs can connect. 0.0.0.0 in bind() means every interface on the machine, so other computers can reach the server too, which is the change Experiment 6 asks for.

7. In the UDP file transfer, why is a sequence number needed? What can happen to UDP datagrams on the way? Because UDP gives no ordering and no delivery guarantee: datagrams can be lost, duplicated, reordered or dropped when a buffer overflows. The 4-byte number lets the receiver put the chunks back in the right order, throw away a repeat and list exactly which ones never arrived. My tests produced Duplicates ignored: 1 and Missing chunks: [7].

8. Why is comparing SHA-256 hashes better than comparing file sizes? Two different files can easily have the same length. A hash depends on every byte, so if even one byte is wrong the value changes completely. The size tells you how many bytes arrived and the hash tells you which ones they were.

9. How is throughput calculated? Why do we average several runs? Throughput in KB/s is (bytes ÷ 1024) ÷ elapsed seconds, where the elapsed time comes from time.perf_counter(). A single run is noisy because of scheduling, caching and other processes; my three TCP runs ranged from 0.0015 s to 0.0023 s, so the average is a fairer number.

10. Why must a file server reject filenames such as ../secret.txt? Because joining it to the shared folder would point outside that folder and let any client read private files on the server. is_valid_name() therefore accepts a plain basename only, and that request is answered with ERROR Invalid filename.

Conclusion

In this lab I built client-server programs with Python sockets over both transport protocols. TCP needed a handshake before anything could be sent and closed with FINs, while UDP sent each message on its own with no setup at all, and Wireshark showed both clearly on the loopback interface. Experiment 4 showed why a receiver cannot assume one recv() equals one send(), which is the reason the file client in Experiment 8 is built around a size header and an exact-length loop. Over UDP I had to add sequence numbers, a chunk count, duplicate detection and a timeout by hand, and the packet-loss test showed what is still missing: TCP repairs loss on its own, my UDP program only reports it. Every transfer was verified with SHA-256, and the timing runs showed that on one machine the measurements say as much about the two programs as about the two protocols.